

Лабораторна робота №2.

Розробка серверного застосунку на Node.js з використанням Express та React

Односторінковий додаток (SPA) – це веб-додаток або веб-сайт, який взаємодіє з користувачем шляхом динамічного переписування поточної вебсторінки з новими даними з веб-сервера, замість того, щоб веб-браузер завантажував цілі нові сторінки. Мета – швидший перехід, завдяки якому вебсайт відчуває себе як при роботі зі звичайними програмами.

У SPA ніколи не відбувається оновлення сторінки; натомість весь необхідний код HTML, JavaScript та CSS браузер отримує або за допомогою одного завантаження сторінки, або відповідні ресурси динамічно завантажуються та додаються на сторінку за необхідності, як правило, у відповідь на дії користувача. Сторінка не перезавантажується в будь-який момент процесу, а також не передає управління на іншу сторінку, хоча хеш розташування або API історії HTML5 можуть бути використані для забезпечення сприйняття та навігації окремих логічних сторінок у програмі.

Node.js представляє середу виконання коду на JavaScript, яка побудована на основі движка JavaScript Chrome V8, який дозволяє транслювати виклики на мові JavaScript в машинний код. Node.js в першу чергу призначений для створення серверних додатків на мові JavaScript. Node.js є відкритим проектом, візидні коди якого можна подивитись на <https://github.com/nodejs>. У Node.js використовується libuv - крос-платформна бібліотека рідтримки з акцентом на асинхронний ввід-вивід.

З точки зору розробника, Node.js однопоточна, алі пед капотом libuv використовує треди, події файлової системи, реалізує цикл подій, включає в себе тред-пулінг і т.д. У більшості випадків ви не будете взаємодіяти з libuv безпосередньо.

1.1 Установка та модульність кода.

Для завантаження потрібно перейти на офіційний сайт <https://nodejs.org/ru/download>. На головній сторінці ви побачите можливі опції для завантаження найостанніша версія і LTS-версія. Рекомендується обирати LTS версію. Ще на офіційному сайті є інші варіанти встановлення, наприклад, на macOS ви можете встановити Node.js за допомогою використання Homebrew або MacPorts.

В Node.js модуль – це набір функцій та об'єктів JavaScript, які можуть використовувати зовнішні програми. Опис частини коду як модуля відноситься не так до самого коду, як до того, що він робить. Будь-який файл або набір Node.js можна вважати модулем, якщо його функції та дані готові до використання зовнішніми програмами.

Оскільки модулі забезпечують функції, які можуть бути використані в більш масштабних програмах, вони дозволяють створювати слабо пов'язані додатки, що масштабуються в міру зростання складності.

Node.js надає можливість розробникам підключати вже створенні модулі, а також створювати свої власні.

Для використання функціональності вже існуючого модуля, вам треба звернутися до нього і підключити до свого проекту, за допомогою команди:

```
const назва_змінної = require(«назва модуля»)
```

Також, можна експортувати не всі можливості модуля, а й окрему функціональність.

Для тестування підключення модулів створіть файл - test.js. Впишіть наступний код, він дозволить використати можливості модуля «os» та отримати ім'я користувача:

```
const os = require('os') // отримаємо ім'я поточного користувача  
let userName = os.userInfo().username console.log(userName)
```

Для того, щоб запустити файл в терміналі необхідно прописати команду node назва файлу. В нашому випадку:

```
node test.js
```

Також, можна створити свій власний модуль і підключити його до вашого проекту. Наприклад, таким чином:

Назва файлу(майбутнього модуля) - sum.js

```
exports=module.exports={}; //визначаємо, що будемо його експортувати в якості  
модуля.  
exports.AddNumber=function(a,b)  
{  
return a+b; };
```

Файл - app.js

```
let Addition=require('./sum.js');  
let result=Addition.AddNumber(1,2); if (result>5){  
console.log("Hi") }  
else { console.log("Bye")  
}
```

Таким чином, ви можете створювати і використовувати вже існуючі модулі.

1.2 Підняття серверу

1.2.1 Створення базового серверу HTTP

Для використання HTTP-сервера та клієнта необхідно `require('node:http')`. Інтерфейси HTTP у Node.js розроблені для підтримки багатьох особливостей протоколу, які традиційно були складними у використанні. Зокрема великі, можливо, закодовані у вигляді шматків повідомлення. Інтерфейс ретельно слідкує за тим, щоб ніколи не буферизувати цілі запити чи відповіді, тому користувач може передавати дані в потоковому режимі.

Простий код створення серверу:

```
const http = require('http') // підключення модуля http  
http.createServer(function (request, response) { //створення серверу, за допомогою JS  
Promise  
  
response.end('Hello NodeJS!') })  
.listen(3000, '127.0.0.1', function () { //початок прослуховування серверу console.log(  
'Сервер почав прослуховування запитів на порту на порту 3000' )  
})
```

Після створення серверу, можна звернутись за вказаною адресою та портом і отримати відображення сторінки(але її в нас ще немає).

1.2.2 Створення серверу за допомогою Express

Express — це мінімальний і гнучкий фреймворк веб-додатків Node.js, який надає надійний набір функцій для розробки веб-додатків і мобільних додатків. Це сприяє швидкому розвитку веб-додатків на основі Node. Нижче наведено деякі з основних функцій Express framework:

- дозволяє налаштувати проміжне програмне забезпечення для відповіді на HTTP-запити;
- визначає таблицю маршрутизації, яка використовується для виконання різних дій на основі методу HTTP та URL-адреси;
- дозволяє динамічно відображати HTML-сторінки на основі передачі аргументів у шаблони;

Для встановлення Express у свій додаток, напишіть у терміналі вашого проекту:

```
npm install express
```

NPM - Node Package Manager, пакетний менеджер, що дозволяє інстальювати необхідні модулі та оточення до своїх додатків.

Після того, як Express буде інстальовано, можна запустити сервер, написавши наступний код в файлі, наприклад, `app.js`.

```
const express = require("express"); const app = express();
app.listen(3000, () => {
console.log("Application started and Listening on port 3000");
});
```

Більшість веб-серверів виконують такі дії:

- прослуховують запити, які надходять на сервер;
- отримують HTTP-запити в кінцевій точці, наприклад /home, і аналізувати інформацію, надіслану разом із запитом;
- виконує код у відповідь на тип заголовка HTTP, з якого було зроблено запит;
- відповідає на запити;

Працюючи з Express, у нас є інструменти, які допоможуть нам виконати ці кроки всього за кілька рядків коду. Три великі концепції, з якими працює Express, це:

- маршрутизація (Routing)
- проміжне програмне забезпечення (Middleware)
- запит/Відповідь (Request/Response).

1.3. Налаштування маршрутизації та get/post запитів за допомогою Express

Маршрутизація має на меті описати, який код потрібно запустити у відповідь на запит, отриманий сервером. Зазвичай це робиться на основі комбінації шаблону URL-адреси та методу HTTP, пов'язаного із запитом.

Наприклад, якщо отримано запит GET для URL-адреси /home, надішліть назад HTML для домашньої сторінки. Або якщо запит POST надіслано до / products, створіть новий список продуктів.

В загальному вигляді виглядає наступним чином:

```
app.get("/", (req, res) => {.....});
```

У функції обробки ми отримуємо два параметри, req і res. Об'єкт req буде містити всю інформацію з отриманого запиту. Ми можемо отримати доступ до властивостей об'єкта req, щоб отримати дані про запит, як-от URL-шлях, тіло запиту, заголовки тощо.

Об'єкт res представляє відповідь, яку ми надсилаємо назад клієнту. Express додає до res корисні методи, які допомагають нам надіслати відповідь. Після надсилання відповіді HTTP-транзакція завершена.

Наприклад, після налаштування маршрутизації файл app.js може виглядати наступним чином:

```
let express = require('express');
let app = express();
```

```
// This responds with "Hello World" on the homepage
app.get('/', function (req, res) {
  console.log("Got a GET request for the homepage"); res.send('Hello GET');
});

// This responds a POST request for the homepage
app.post('/', function (req, res) {
  console.log("Got a POST request for the homepage"); res.send('Hello POST');
});

// This responds a GET request for the /list_user page.
app.get('/list_user', function (req, res) { console.log("Got a GET request for
/list_user"); res.send('Page Listing');
});
app.listen(3000, () => {
  console.log("Application started and Listening on port 3000");
});
```

детальніше про маршрутизацію, можна почитати ось тут - <https://expressjs.com/en/guide/routing.html>.

1.4 Налаштування Front-End частини

Так як Node - це серверна частина веб-додатків, є декілька варіантів налаштування Front-чвстини для вашого додатку.

Перший варіант підвантаження html-документів безпосередньо із файлової системи вашого комп'ютера, це ми можемо зробити за допомогою модуля fs. Наприклад, ось таким чином, модифікувавши файл app.js:

```
app.get("/", (req, res) => { res.send("<h1>Hello world2!</h1>");
});
app.get("/contacts", (req, res) => { res.sendFile(__dirname + '/index.html');
});
```

Цей код дозволяє нам, звернувшись до <http://127.0.0.1:3000/> Отримати на екрані - hello world!

А звернувшись за адресою <http://127.0.0.1:3000/contacts> підтягнути і відобразити сторінки index.html.

Другий варіант налаштування фронт-частини це використання шаблонів, та шаблонізаторів. Наприклад, moustache, handleBars та інші.

Третій варіант - використовувати react, angular або інший фреймворк для організації роботи із фронт-частиною.

Рівень 1

1. Створіть простий блог на Node.js з використанням Express. Реалізуйте фронтенд на React для відображення статей з блогу.
2. Створіть просту систему управління задачами на Node.js з використанням Express. Реалізуйте фронтенд на React для відображення списку задач.
3. Створіть простий каталог товарів на Node.js з використанням Express. Реалізуйте фронтенд на React для відображення товарів.
4. Створіть базову соціальну мережу на Node.js з використанням Express. Реалізуйте фронтенд на React для відображення користувачів та їх постів.
5. Створіть простий каталог курсів на Node.js з використанням Express. Реалізуйте фронтенд на React для відображення курсів.
6. Створіть простий відеохостинг на Node.js з використанням Express. Реалізуйте фронтенд на React для відображення відео.
7. Створіть просту онлайн-гру на Node.js з використанням Express. Реалізуйте фронтенд на React для гри.
8. Створіть простий музичний стрімінг на Node.js з використанням Express. Реалізуйте фронтенд на React для відображення треків.

Рівень 2

1. Додайте можливість додавання нових статей та коментарів. Реалізуйте систему категорій для статей.
2. Додайте можливість додавання, редагування та видалення задач. Реалізуйте функцію перегляду деталей кожної задачі.
3. Додайте можливість додавання товарів у кошик. Реалізуйте систему оформлення замовлення.
4. Додайте можливість створювати та коментувати пости. Реалізуйте систему дружніх зв'язків між користувачами.
5. Додайте можливість запису на курс. Реалізуйте систему відгуків та рейтингу курсів.
6. Додайте можливість завантаження та перегляду відео. Реалізуйте систему коментування відео.
7. Додайте можливість множинного підключення користувачів до гри. Реалізуйте систему лідерборду.
8. Додайте можливість прослуховування та завантаження музики. Реалізуйте систему плейлистів та поділу музики за жанрами.

Рівень 3

1. Додайте можливість авторизації користувачів. Реалізуйте систему пошуку та фільтрації статей.
2. Реалізуйте систему користувачів та ролей доступу. Додайте можливість планування та нагадувань.
3. Реалізуйте систему користувачів, авторизації та аутентифікації. Додайте систему відстеження статусу замовлення.
4. Реалізуйте систему особистих повідомлень між користувачами. Додайте систему пошуку користувачів та постів.
5. Реалізуйте систему користувачів, авторизації та аутентифікації. Додайте систему створення та відстеження навчальних програм.
6. Реалізуйте систему користувачів та управління їхніми правами. Додайте функціонал "Поділитися відео", щоб користувачі могли надсилати відео іншим користувачам.
7. Додайте систему збереження стану гри та історії матчів.
8. Реалізуйте систему користувачів та управління їхніми профілями. Додайте можливість створення та відстеження відгуків до треків для кожного користувача.

Рівень 4

1. Впроваджуйте кешування на сервері для покращення швидкодії. Реалізуйте кешування результатів запитів або інше кешування за необхідності.
2. Інтегруйте систему з електронною поштою для надсилання нагадувань. Реалізуйте аналітику виконання задач та звітність.
3. Оптимізуйте швидкість завантаження сторінок. Реалізуйте систему рекомендацій на основі покупок користувачів.
4. Використовуйте компресію (gzip) для передачі даних з сервера на клієнтський інтерфейс для зменшення об'єму даних, які передаються по мережі.
5. Розширте функціонал вашого клієнтського інтерфейсу, додаючи маршрути та компоненти React для відображення інших типів об'єктів чи даних. Забезпечте відповідні зміни на серверній стороні для підтримки цих нових функціональностей.
6. Оптимізуйте систему для високих навантажень. Реалізуйте систему підписки на канали та повідомлень про нові відео.
7. Оптимізуйте гру для високих навантажень. Реалізуйте систему групових матчів та турнірів.
8. Інтегруйте систему аналітики для відстеження використання медіа-файлів користувачами.

Контрольні запитання

1. Як ви організували структуру вашого проекту на Node.js?

2. Які основні завдання виконує серверний застосунок у вашій системі?
3. Як ви інтегрували бібліотеку Express в ваш серверний застосунок?
4. Які маршрути (routes) були реалізовані у вашому серверному застосунку? Наведіть приклади.
5. Як ви організували взаємодію між сервером і базою даних?
6. Які моделі даних були використані у вашому застосунку?
7. Як ви забезпечили обробку помилок на сервері?
8. Як ви реалізували аутентифікацію та авторизацію користувачів?
9. Як ви використовували стан (state) у вашому React-фронтенді?
10. Як ви організували реактивне оновлення інтерфейсу під час зміни стану?
11. Як ви інтегрували ваш React-фронтенд з серверним застосунком?
12. Які компоненти React ви використали у вашому застосунку?
13. Як ви реалізували маршрутизацію на фронтенді за допомогою React?
14. Як ви організували управління формами в вашому React-фронтенді?
15. Як ви реалізували валідацію даних на клієнтському та серверному рівнях?
16. Як ви забезпечили створення, редагування та видалення ресурсів в вашому застосунку?
17. Як ви організували оптимізацію завантаження сторінок у вашому React-застосунку?
18. Як ви інтегрували зовнішні API або сервіси у ваш застосунок?
19. Як ви тестували ваш серверний застосунок та React-фронтенд?
20. Які основні виклики ви зустріли під час розробки і як ви їх подолали?